Process & Decision Documentation

Entry Header

Name: Yukie Fan

Role(s): Sole Creator

Primary responsibility for this work: To edit the given template and add a level to the example given.

Goal of Work Session

The goal of this work session is to modify the camera state's code to evoke a certain emotion.

GenAI Documentation

Date Used: Feb. 23rd, 2026

Tool Disclosure: CoPilot GPT -5.1

Purpose of Use: To clarify side quest task and identify which parameters needed to be edited to complete the task.

Summary of Interaction: I asked the AI to explain to me how I can compete the task and parameters I needed to change to do that.

Human Decision Point(s): I realized the answer they gave me were a bit vague on where to add new codes.

Integrity & Verification Note: I looked back at the Side Quest outline and asked peers on clarification.

Scope of GenAI Use: GenAI did not help with my ReadMe document, Process and Documentation pdf, and the generation of which emotion I wanted to evoke.

Limitations or Misfires: I had to prompt it a couple times to tell me where I should be inserting the code.

Summary of Process (Human + Tool)

First, I read through the side quest prompt and was confused/wanted more clarification. I turned to AI and inputted the sketch.js code into Copilot to see if they could define my task. I also asked it which parameters I needed to change for me to complete my task. Upon

reading the response, my task was clearer, however I felt the answer they provided was a bit wrong, so I asked them to point out which lines of code. I realized that I needed to change the code for the Camera2D.js file so I pasted the code for that into my question. The AI was able to more accurately determine what I needed to change in order to evoke my chosen emotion.

Decision Points & Trade-offs

Some decisions I made was to ask the AI for more clarification on where to insert the new code and if this code accurately reflected on the emotion I chose.

Verification & Judgement

I cross referenced with peers and lecture notes to ensure that I was doing the side quest correctly. I also made sure that my code was correct via LiveServer, seeing my code live.

Appendix

Me: Create a reflective or meditative camera experience that scrolls through a world larger than the screen. Use pacing and motion to evoke emotion. I want to evoke the dream-like emotion.

/* Week 5 — Example 5: Side-Scroller Platformer with JSON Levels + Modular Camera

Course: GBDA302 Instructors: Dr. Karen Cochrane & David Han Date: Feb. 12, 2026

Move: WASD/Arrows Jump: Space

Learning goals:

- Build a side-scrolling platformer using modular game systems
- Load complete level definitions from external JSON (LevelLoader + levels.json)
- Separate responsibilities across classes (Player, Platform, Camera, World)
- Implement gravity, jumping, and collision with platforms
- Use a dedicated Camera2D class for smooth horizontal tracking
- Support multiple levels and easy tuning through data files
- Explore scalable project architecture for larger games */

const VIEW_W = 800; const VIEW_H = 480;

```
let allLevelsData; let levelIndex = 0;

let level; let player; let cam;

function preload() { allLevelsData = loadJSON("levels.json"); // levels.json beside index.html [web:122] }

function setup() { createCanvas(VIEW_W, VIEW_H); textFont("sans-serif"); textSize(14);

cam = new Camera2D(width, height); loadLevel(levelIndex); }

function loadLevel(i) { level = LevelLoader.fromLevelsJson(allLevelsData, i);

player = new BlobPlayer(); player.spawnFromLevel(level);

cam.x = player.x - width / 2; cam.y = 0; cam.clampToWorld(level.w, level.h); }

function draw() { // --- game state --- player.update(level);

// Fall death → respawn if (player.y - player.r > level.deathY) { loadLevel(levelIndex); return; }

// --- view state (data-driven smoothing) --- cam.followSideScrollerX(player.x, level.camLerp); cam.y = 0; cam.clampToWorld(level.w, level.h);

// --- draw --- cam.begin(); level.drawWorld(); player.draw(level.theme.blob); cam.end();

// HUD fill(0); noStroke(); text(level.name + " (Example 5)", 10, 18); text("A/D or ←/→ move • Space/W/↑ jump • Fall = respawn", 10, 36); text("camLerp(JSON): " + level.camLerp + " world.w: " + level.w, 10, 54); text("cam: " + cam.x + ", " + cam.y, 10, 90); const p0 = level.platforms[0]; text(p0: x=${p0.x} y=${p0.y} w=${p0.w} h=${p0.h}, 10, 108);

text( "platforms: " + level.platforms.length + " start: " + level.start.x + "," + level.start.y, 10, 72, ); }

function keyPressed() { if (key === " " || key === "W" || key === "w" || keyCode === UP_ARROW) { player.tryJump(); } if (key === "r" || key === "R") loadLevel(levelIndex); }
```

GenAI: If you want this to feel *dream-like*, we can push the camera from “gentle drift” into something that feels **floaty, surreal, and slightly impossible**—like the world is breathing around you rather than scrolling past you.

Below is a refined concept and a drop-in code block that transforms your camera into a drifting, dream-logic observer.

🌙 Dream-Like Camera Philosophy

A dream doesn't move in straight lines. It *wanders*. It *glides*. It *forgets* where it was going and then suddenly remembers.

To evoke that feeling, we use:

1. Nonlinear motion

- Layered sine waves
- Slow Perlin noise
- Occasional "tilts" or micro-accelerations

2. Soft unpredictability

Nothing should feel random, but nothing should feel mechanical either.

3. Weightless vertical drift

Dreams rarely stay locked to the ground.

4. Parallax that feels like memory

Background layers move at different speeds, as if they're fading in and out of attention.

🌫️ DreamScroll Mode (drop-in camera update)

This version is more surreal than the meditative one—motion overlaps, layers drift, and the camera feels like it's floating through a half-remembered place.

```
class Camera2D {
 constructor(w, h) {
  this.viewW = w;
  this.viewH = h;

  this.x = 0;
  this.y = 0;

  this.time = 0;
```

```
    this.baseSpeed = 0.35; // slow dream drift
  }

  updateDream(dt = 1) {
    this.time += dt * 0.01;

    // --- horizontal drift (layered waves) ---
    const wave1 = sin(this.time * 0.4) * 0.6;
    const wave2 = sin(this.time * 1.3) * 0.2;
    const wander = noise(this.time * 0.25) * 0.5;

    this.x += this.baseSpeed + wave1 + wave2 + wander;

    // --- vertical float (dreamlike buoyancy) ---
    const float1 = sin(this.time * 0.7) * 25;
    const float2 = noise(this.time * 0.5) * 15;

    this.y = float1 + float2;

    // --- occasional “tilt” (subtle acceleration moments) ---
    if (noise(this.time * 0.1) > 0.75) {
      this.x += 1.2; // a soft push forward
    }
  }
}
```

This camera:

- Never moves the same way twice
- Feels like drifting through a memory
- Has moments of stillness and moments of sudden glide
- Creates a sense of depth even without parallax layers

🌌 How to use it in your draw loop

Replace your normal follow logic:

```
cam.updateDream();

cam.clampToWorld(level.w, level.h);
```

If you want to toggle modes:

```
let dreamMode = true;


function keyPressed() {
  if (key === "m" || key === "M") {
    dreamMode = !dreamMode;
  }
}
```

Then:

```
if (dreamMode) {
  cam.updateDream();
} else {
  cam.followSideScrollerX(player.x, level.camLerp);
}
```

🌒 Optional Dream Enhancements

If you want to push the dream feeling even further, I can help you add:

1. Parallax fog layers

Soft drifting shapes that move slower than the world.

2. Firefly particles

Tiny glowing dots that float with the camera.

3. Color shifting

Hue rotation based on camera time.

4. Memory echoes

Ghost-images of platforms fading in/out.

5. Slow-motion pockets

Areas where time dilates as the camera enters them.

If you tell me the *emotion* you want—calm, uncanny, nostalgic, euphoric—I can tune the camera motion and world behavior to match it.

Me: how would i code this into my code?

GenAI: Absolutely — let's wire this directly into **your existing codebase** so it works cleanly with your current Camera2D, draw(), and level-loading structure.

I'll show you **exactly where to paste things** and **what to replace**, so you don't have to guess.

✅ STEP 1 — Add the Dream Camera Method to Your Camera2D Class

Find your Camera2D class (wherever it lives in your project) and **add this method inside it**:

```
updateDream(dt = 1) {
 this.time += dt * 0.01;


 // horizontal drift (layered waves)
 const wave1 = sin(this.time * 0.4) * 0.6;
 const wave2 = sin(this.time * 1.3) * 0.2;
```

```
  const wander = noise(this.time * 0.25) * 0.5;

  this.x += this.baseSpeed + wave1 + wave2 + wander;

  // vertical float
  const float1 = sin(this.time * 0.7) * 25;
  const float2 = noise(this.time * 0.5) * 15;

  this.y = float1 + float2;

  // occasional dream "push"
  if (noise(this.time * 0.1) > 0.75) {
    this.x += 1.2;
  }
}
```

If your class doesn't have this.time or this.baseSpeed, add these in the constructor:

```
this.time = 0;
this.baseSpeed = 0.35;
```

✅ STEP 2 — Add a Toggle Variable in Your Main Sketch

At the top of your file (near your globals):

```
let dreamMode = true;
```

✅ STEP 3 — Modify Your draw() Loop

Right now you have:

```
cam.followSideScrollerX(player.x, level.camLerp);
```

```
cam.y = 0;
```

Replace that block with:

```
if (dreamMode) {
  cam.updateDream();
} else {
  cam.followSideScrollerX(player.x, level.camLerp);
  cam.y = 0;
}
```

Then keep your existing:

```
cam.clampToWorld(level.w, level.h);
```

✅ STEP 4 — Add a Key to Toggle Dream Mode

Inside your existing keyPressed():

```
if (key === "m" || key === "M") {
  dreamMode = !dreamMode;
}
```

This lets you switch between:

- **Dream camera**
- **Normal side-scroller camera**

🎉 That's it — your game now has a dream-like camera

When dreamMode = true:

- The camera drifts on its own
- The player no longer controls the view
- The world feels floaty, surreal, and alive
- Vertical motion becomes soft and buoyant

- Occasional “pushes” make the scroll feel dream-logic

Your world becomes a **moving dreamscape** instead of a platformer.

If you want, I can help you next with:

🌫️ **Parallax fog layers**

✨ **Fireflies / drifting particles**

🎨 **Color-shifting dream palettes**

🔁 **Infinite looping dream worlds**

🎵 **Camera motion synced to ambient music**

Just tell me the vibe you want to lean into.

Me: class Camera2D { constructor(viewW, viewH) { this.viewW = viewW; this.viewH = viewH; this.x = 0; this.y = 0; }

followSideScrollerX(targetX, lerpAmt) { const desired = targetX - this.viewW / 2; this.x = lerp(this.x, desired, lerpAmt); }

clampToWorld(worldW, worldH) { const maxX = max(0, worldW - this.viewW); const maxY = max(0, worldH - this.viewH); this.x = constrain(this.x, 0, maxX); this.y = constrain(this.y, 0, maxY); }

begin() { push(); translate(-this.x, -this.y); } end() { pop(); } }

GenAI: Perfect — thanks for sharing your current Camera2D class.
Let’s integrate the dream-like camera **directly into this exact class** with zero guesswork.

Below is a clean, drop-in upgrade that keeps everything you already have and simply *adds* the dream-scroll behavior.

✅ 1. Add Dream Variables to Your Constructor

Your current constructor:

constructor(viewW, viewH) {

 this.viewW = viewW;

```
  this.viewH = viewH;
  this.x = 0;
  this.y = 0;
}
```

Modify it to include the dream camera state:

```
constructor(viewW, viewH) {
  this.viewW = viewW;
  this.viewH = viewH;
  this.x = 0;
  this.y = 0;

  // Dream camera state
  this.time = 0;
  this.baseSpeed = 0.35; // slow drifting speed
}
```

✅ 2. Add the Dream Camera Method Inside the Class

Paste this **inside the Camera2D class**, below your existing methods:

```
updateDream(dt = 1) {
  this.time += dt * 0.01;

  // Horizontal drift (layered waves + noise)
  const wave1 = sin(this.time * 0.4) * 0.6;
  const wave2 = sin(this.time * 1.3) * 0.2;
  const wander = noise(this.time * 0.25) * 0.5;
```

```
  this.x += this.baseSpeed + wave1 + wave2 + wander;

  // Vertical float (dreamlike buoyancy)
  const float1 = sin(this.time * 0.7) * 25;
  const float2 = noise(this.time * 0.5) * 15;

  this.y = float1 + float2;

  // Occasional dream "push"
  if (noise(this.time * 0.1) > 0.75) {
    this.x += 1.2;
  }
}
```

This method does not break anything in your existing game — it simply gives you a new camera mode.

✅ 3. Modify Your draw() Loop to Use the Dream Camera

Find this part of your draw loop:

```
cam.followSideScrollerX(player.x, level.camLerp);
cam.y = 0;
```

Replace it with:

```
if (dreamMode) {
  cam.updateDream();
} else {
  cam.followSideScrollerX(player.x, level.camLerp);
  cam.y = 0;
```

```
}
```

Keep your existing:

```
cam.clampToWorld(level.w, level.h);
```

✅ 4. Add a Toggle Variable

At the top of your sketch:

```
let dreamMode = true;
```

✅ 5. Add a Key to Toggle Dream Mode

Inside keyPressed():

```
if (key === "m" || key === "M") {
  dreamMode = !dreamMode;
}
```

🌙 Now Your Camera Has Two Modes

Dream Mode (M key toggles it)

- Camera drifts on its own
- World feels floaty and surreal
- Player no longer controls the view
- Vertical motion becomes soft and buoyant
- Occasional dream-logic accelerations

Normal Mode

- Camera follows the player like before

If you want, I can help you next with:

- Adding **parallax fog layers**

- Adding **fireflies / drifting particles**
- Making the **world loop infinitely**
- Creating a **dream-like color palette shift**

Just tell me the vibe you want to push toward.

Me: where do i add these codes?

GenAI: Got you — let's make this super clear and practical.
Here's **exactly where each piece goes** inside *your* project structure.

I'll walk you through it step-by-step so you can paste everything in the right place without breaking anything.

✅ 1. Modify Your Camera2D Class (the file you just pasted)

You will **edit the class you showed me**.

A. Add dream variables to the constructor

Find this:

```
constructor(viewW, viewH) {
  this.viewW = viewW;
  this.viewH = viewH;
  this.x = 0;
  this.y = 0;
}
```

Replace it with:

```
constructor(viewW, viewH) {
  this.viewW = viewW;
  this.viewH = viewH;
  this.x = 0;
  this.y = 0;
```

```
  // Dream camera state
  this.time = 0;
  this.baseSpeed = 0.35;
}
```

B. Add the dream camera method INSIDE the class

Paste this **right below your existing methods**, but still inside the class:

```
updateDream(dt = 1) {
  this.time += dt * 0.01;

  // Horizontal drift (layered waves + noise)
  const wave1 = sin(this.time * 0.4) * 0.6;
  const wave2 = sin(this.time * 1.3) * 0.2;
  const wander = noise(this.time * 0.25) * 0.5;

  this.x += this.baseSpeed + wave1 + wave2 + wander;

  // Vertical float (dreamlike buoyancy)
  const float1 = sin(this.time * 0.7) * 25;
  const float2 = noise(this.time * 0.5) * 15;

  this.y = float1 + float2;

  // Occasional dream “push”
  if (noise(this.time * 0.1) > 0.75) {
```

```
    this.x += 1.2;
  }
}
```

Your full class will now look like:

```
class Camera2D {
  constructor(...) { ... }

  followSideScrollerX(...) { ... }

  clampToWorld(...) { ... }

  begin() { ... }

  end() { ... }

  updateDream(...) { ... }  <-- NEW
}
```

✅ 2. Add a toggle variable at the top of your main sketch

In your main .js file (the one with setup() and draw()):

```
let dreamMode = true;
```

Put it near your other globals:

```
let allLevelsData;
let levelIndex = 0;

let level;
```

```
let player;
let cam;

let dreamMode = true; // <--- add this
```

✅ 3. Modify your draw() loop

Find this part:

```
cam.followSideScrollerX(player.x, level.camLerp);
cam.y = 0;
cam.clampToWorld(level.w, level.h);
```

Replace it with:

```
if (dreamMode) {
  cam.updateDream();
} else {
  cam.followSideScrollerX(player.x, level.camLerp);
  cam.y = 0;
}

cam.clampToWorld(level.w, level.h);
```

This is the **exact spot** where the dream camera takes over.

✅ 4. Add a key to toggle dream mode

Inside your existing keyPressed():

```
if (key === "m" || key === "M") {
  dreamMode = !dreamMode;
}
```

You can put it right under your jump key logic.

🎉 **Now your game supports dream-mode camera scrolling**

When dreamMode = true:

- Camera drifts on its own
- Player no longer controls the view
- Vertical floating + layered waves
- Occasional dream-logic pushes
- The world feels surreal and alive

When dreamMode = false:

- Camera behaves exactly like before

If you want, I can help you next with:

✨ **Fireflies**

🌫️ **Parallax fog**

🎨 **Color-shifting dream palettes**

🔁 **Infinite looping dream worlds**

🎵 **Camera synced to ambient music**

Just tell me what direction you want the dream to go.